

Clothing Classification on Fashion-MNIST

Decision trees, tree ensembles and a deep learning design

Thijs Niekerk (SNR 2125613)

Pjotr Martens (SNR 2080478)

Machine Learning Analytics, Final Project

Tilburg School of Economics and Management, Tilburg University

October 2026

1 Data and set-up

Fashion-MNIST (Xiao et al., 2017) has 70,000 grayscale 28×28 images of ten balanced clothing classes (60,000 for training, 10,000 for testing). We flatten each image into 784 pixel values and scale them to $[0, 1]$. The average images of the classes (Figure 1) show that Pullover, Coat and Shirt have almost the same shape (their mean images correlate 0.95–0.97), with T-shirt closest to Shirt (0.87). Labelling a test image by its closest average image gives 67.7% accuracy, but an F1 score of only 0.26 for Shirt. k-means with ten clusters (Figure 5) puts at least 84% of the Trouser, Sneaker and Ankle boot images in one cluster each, but 47% of the T-shirt, Pullover, Coat and Shirt images land in a cluster dominated by one of the other three, so we expect these upper-body classes to be the hard part. Finally, neighbouring pixels are strongly related (horizontal neighbours correlate 0.78, pixels seven columns apart 0.32), and 105 border pixels are zero in over 90% of the images. Trees are not affected by rescaling and rarely split on empty pixels, so we do no further preprocessing.

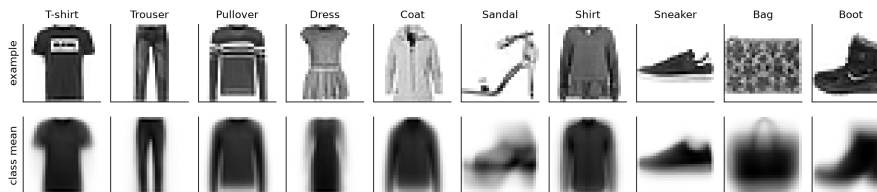


Figure 1: One training image (top) and the average training image (bottom) of every class.

All hyperparameters are tuned with 3-fold cross-validation (CV) on the 60,000 training images. Final models are refitted on all training images and every model is evaluated once on the same untouched test set. Next to accuracy, which is fair with balanced classes, we use per-class F1 scores and confusion matrices.

2 Part 1: decision tree and tree ensemble

2.1 Decision tree (1a)

We use `DecisionTreeClassifier` (Pedregosa et al., 2011) with the Gini index and compare two stopping criteria (maximum depth, minimum images per leaf) as well as cost-complexity pruning. An unrestricted tree fits the training data perfectly but reaches only 78.9% in CV (Figure 2a): it overfits. A depth of 12 with at least 10 images per leaf gives 81.2%; pruning with $\alpha = 2 \cdot 10^{-4}$ does slightly better (81.5%), so we use the pruned tree (270 leaves), which reaches 80.8% test accuracy (Table 1). Its first split asks whether one pixel near the top of the image is empty, which separates footwear (and some bags) from clothing.

The errors are very uneven (Figure 3a, Table 3). Trouser, Bag, Sandal and Ankle boot reach F1 scores above 0.90, but Shirt (0.54), Coat (0.67) and Pullover (0.68) do not, and 56% of all mistakes are confusions among the four upper-body classes. Only 499 of the 1,000 test shirts are recognised, as Section 1 suggested. A tree only checks single pixels against a threshold, which works when one pixel is decisive (the gap between trouser legs) but not when classes differ in a collar or buttons that sit in a slightly different place in every image. The tree thus has two problems: *high variance* (the gap between training and CV accuracy) and a *bias* from looking at single raw pixels.

2.2 Ensemble method (1b)

Choice and motivation. Tree ensembles come in two kinds. Bagging trains many deep trees independently and averages them, which mainly reduces variance; boosting adds small trees one by one, each correcting the errors of the previous ones, which mainly reduces bias. Section 2.1 showed that our single decision tree mainly suffers from variance: grown fully it fits the training data perfectly (100%) but reaches only 78.9% in CV. Bagging therefore fits our problem better than boosting, which would mostly attack the smaller bias problem, overfits as the number of

rounds grows unless the learning rate and number of rounds are tuned carefully, and has to build its trees one after the other. We use `GradientBoostingClassifier` with default settings only as a reference.

Within bagging we choose a **random forest** (Breiman, 2001). Averaging trees does not change the bias and never increases the variance, but how much it lowers the variance depends on how correlated the trees are. With 784 correlated pixels, plain bagged trees tend to use the same strong pixels near the root and become very similar. A random forest considers only a random subset of $m < p$ pixels at every split, so a strong pixel cannot dominate most trees and the trees become less correlated.

Implementation. We use `RandomForestClassifier` with fully grown trees and first tune the number of trees with $m = \sqrt{p} = 28$: CV accuracy rises quickly up to 50 trees (87.7%) and only slowly after that (88.1% at 200, Figure 2b), and more trees cannot cause overfitting, so the final forest has 200 trees (on average 38 levels deep, about 5,000 leaves). Because 50 trees are already enough to compare settings, and much faster, we then vary m from 7 to 112 with 50 trees each and compare with plain bagging ($m = p = 784$, `BaggingClassifier`) of the same size (Figure 2c). CV accuracy rises steadily from 86.9% at $m = 7$ via 87.7% at $m = 28$ to 87.9% at $m = 112$, but drops to 87.1% for plain bagging. The single trees show why: with $m = 7$ they are weak (73.0% accuracy) but their errors are less correlated (0.33), bagged trees are stronger (77.6%) but more correlated (0.41), and $m = 28$ lies in between (75.9%, 0.36). Since $m = 112$ beats $m = 28$ by only 0.3 points, about the spread between folds, while each split takes 4 times as long, we keep the usual $m = \sqrt{p} = 28$.

Model	Test acc.	Macro F1	Shirt F1	With noise
Closest average image (baseline)	67.7%	0.672	0.26	–
Decision tree (pruned)	80.8%	0.807	0.54	45.9%
Random forest (200 trees)	87.8%	0.877	0.65	77.1%
Gradient boosting (reference)	86.7%	0.866	0.64	70.9%

Table 1: Test results. *With noise:* Gaussian noise ($\sigma = 0.1$) added to the test images.

Expected versus observed effects. We expected the forest to remove most of the variance but not the pixel-level bias, so the same classes should stay difficult, and this is what we found. Accuracy rises from 80.8% to 87.8%: the forest corrects 965 of the tree’s mistakes and makes 263 new ones. Every class improves (Shirt F1 0.54→0.65), but the share of errors among the upper-body classes grows from 56% to 65% (Figure 3b): the forest mostly removed the scattered mistakes. To test robustness, for example to lower-quality photos, we add Gaussian noise ($\sigma = 0.1$) to every test pixel: the tree drops to 45.9%, since splits such as its first one test whether a pixel is empty, while the forest, which averages over many pixels, keeps 77.1%. SHAP values from `TreeExplainer` on 200 held-out images and the impurity-based importance agree (correlation 0.91 for the same forest, Figure 4): the forest relies on outline pixels at the neckline, the sleeves and the gap between trouser legs. On the first CV fold, a forest on only the 400 highest-SHAP pixels does as well as one on all pixels (87.4% against 87.3%), and 100 pixels still give 84.9%. The pixels pushing a shirt towards Shirt are spread thinly over neckline and hem (Figure 4c–d). Gradient boosting with default settings reaches 86.7%, slightly below the forest; its training accuracy (90.3%) is close to its test accuracy and both still rose after 100 rounds (Figure 6), so its small depth-3 trees were still reducing bias. With noise it falls to 70.9%: like the tree it relies on single-pixel splits, without averaging over independent trees.

Difficulties. `GradientBoostingClassifier` fits ten trees per round (one per class) on 784 pixels and took 89 minutes, so we could not tune it. SHAP values are slow for deep trees, so we used 200 held-out images and a 50-tree forest; as a forest’s SHAP values average over its trees, this gives nearly the same map.

3 Part 2: a deep learning approach

3.1 Shortcomings of the tree-based approach (2c)

(i) **No use of image structure.** Every split looks at one pixel, so the correlation between neighbours is ignored and a collar needs a long chain of splits. (ii) **No position invariance.** What is learned about one pixel does not carry over to its neighbour, so a collar two pixels further left must be learned again. (iii) **Remaining bias.** Beyond about 50 trees accuracy hardly improves, yet 65% of the remaining mistakes are upper-body confusions and boosting, which targets bias, was still improving after 100 rounds. The model needs better features, not more trees.

3.2 Proposed design: a convolutional neural network (2d)

These shortcomings lead us to a **convolutional neural network** (CNN) (LeCun et al., 1998), which we would build in Keras (Table 2). Small 3×3 filters on the 28×28 image learn local patterns such as edges and collar shapes, using the strong correlation between neighbouring pixels that the trees ignore (i). Each filter slides over the whole image, so a pattern is found wherever it appears, and max pooling per 2×2 region makes the output less sensitive to small shifts (ii). The second convolution combines these patterns into larger parts such as sleeves and collars, the higher-level features the forest could not build (iii). Pooling twice, to 7×7 , halves the dense layer compared with a single block. Against overfitting we use dropout (which acts like an ensemble, as in our forest), an L2 penalty, early stopping and data augmentation with class-preserving flips and two-pixel shifts, which teaches shift invariance (ii). A fully connected network or an autoencoder works on flattened pixels and keeps shortcomings (i) and (ii); a network on the pooled 14×14 image loses the small details that separate Shirt from Pullover.

Layer (Keras)	Output shape	Parameters
Conv2D(32, 3×3 , ReLU, padding same), MaxPooling2D(2×2)	$14 \times 14 \times 32$	320
Conv2D(64, 3×3 , ReLU, padding same), MaxPooling2D(2×2)	$7 \times 7 \times 64$	18,496
Flatten, Dense(150, ReLU, L2 penalty), Dropout(0.5)	150	470,550
Dense(10, softmax)	10	1,510
Total		490,876

Table 2: Proposed CNN (augmentation layers in front, active only in training, are not shown).

3.3 Implementation and expected results (2e)

We reshape the images to $28 \times 28 \times 1$, one-hot encode the labels and compile with the Adam optimiser and categorical cross-entropy loss. We train with batch size 32 for at most 30 epochs, hold out a stratified 10% of the training images for validation and stop once the validation loss has not improved for three epochs, keeping the best weights. Dropout, L2 weight and filter counts are tuned on the validation set, using the training curves to spot overfitting. We test once with the measures of Table 1 and compare with a fully connected network (784–150–60–10) trained the same way.

We expect about 92% test accuracy: eight of the ten two-block CNNs in the public benchmark reach 91.6–93.9% (Zalando Research, 2026). Most of the gain should go to Shirt, Pullover and Coat, but Shirt will stay the weakest class, as some images are ambiguous even for people (83.5% human accuracy, Zalando Research, 2026). A single untuned run of this design (Appendix A) reaches 90.7% (Shirt F1 0.74) against 86.6% for a fully connected network; tuning, which we did not do, should close most of the gap to the benchmark.

4 Conclusion

A random forest removes most of the tree’s variance (37% fewer errors, far more robust to noise); the remaining upper-body confusions are bias, which our proposed CNN targets.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Xiao, H., Rasul, K. and Vollgraf, R. (2017). Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*.
- Zalando Research (2026). Fashion-MNIST repository, benchmark table. <https://github.com/zalando-research/fashion-mnist>, accessed October 2026.

A Figures and tables

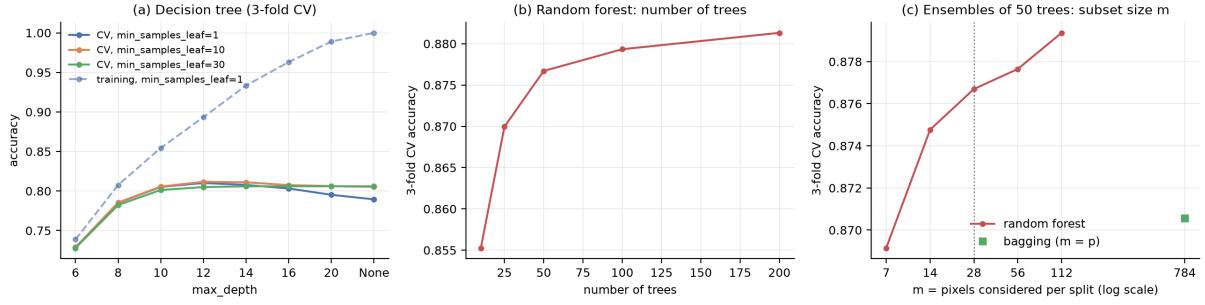


Figure 2: Model selection. (a) Decision tree: 3-fold CV accuracy against `max_depth` for three values of `min_samples_leaf`, and training accuracy for `min_samples_leaf`=1. (b) Random forest ($m = 28$): 3-fold CV accuracy against the number of trees. (c) Ensembles of 50 trees: 3-fold CV accuracy against the number m of pixels considered per split; the square is bagging ($m = p$), the dotted line the chosen $m = \sqrt{p}$.

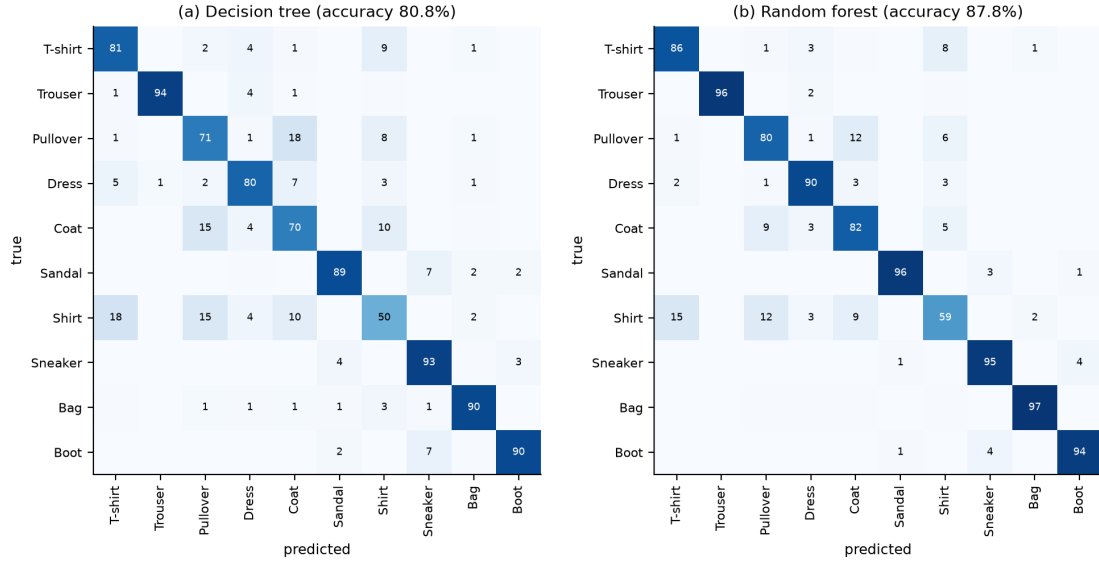


Figure 3: Row-normalised test confusion matrices (% of each true class; entries below 1% left blank).

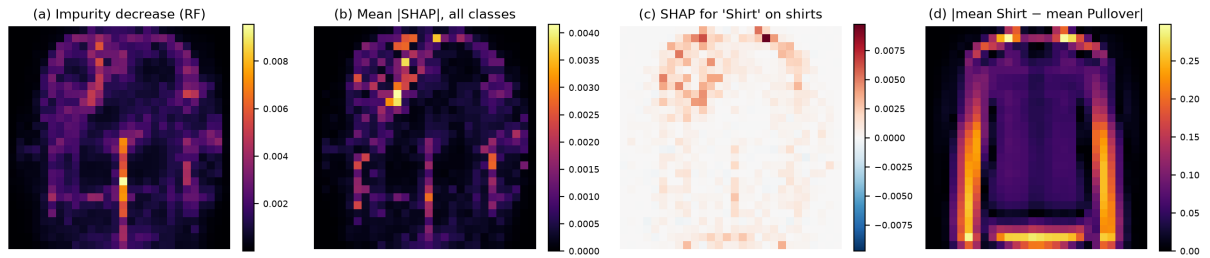


Figure 4: (a) Impurity-based importance of the final 200-tree random forest. (b) Mean absolute SHAP value over 200 held-out images and all classes (50-tree forest). (c) Mean SHAP value for the class Shirt on the true shirts among these images (red pushes towards Shirt). (d) Absolute difference between the mean Shirt and mean Pullover images.

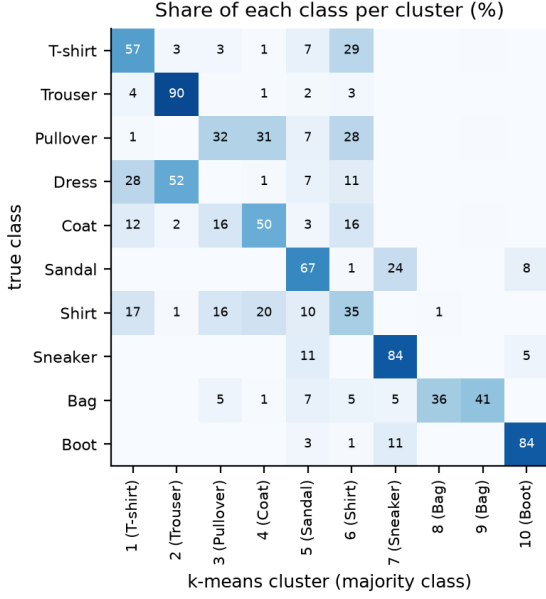


Figure 5: k-means with ten clusters on the training images: share of every class in each cluster.

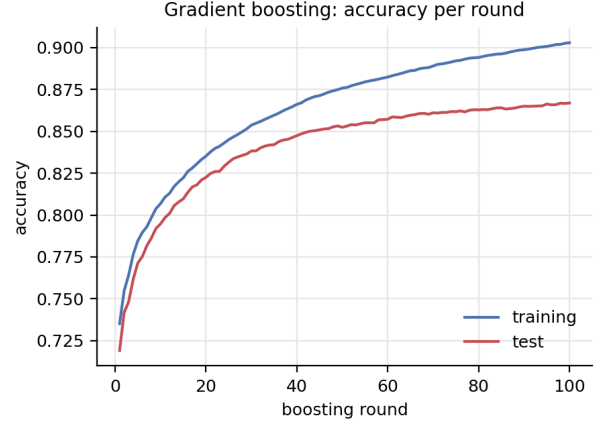


Figure 6: Gradient boosting (default settings, all training data): training and test accuracy after every round.

Class	Avg. image	Tree	Random forest	Gradient boosting	CNN check
T-shirt/top	0.698	0.785	0.843	0.823	0.851
Trouser	0.924	0.956	0.980	0.977	0.985
Pullover	0.491	0.685	0.784	0.767	0.863
Dress	0.726	0.802	0.892	0.874	0.907
Coat	0.538	0.673	0.793	0.774	0.844
Sandal	0.606	0.904	0.968	0.962	0.977
Shirt	0.263	0.541	0.649	0.642	0.738
Sneaker	0.790	0.897	0.940	0.937	0.955
Bag	0.825	0.907	0.967	0.959	0.986
Ankle boot	0.864	0.921	0.949	0.946	0.965
Macro F1	0.672	0.807	0.877	0.866	0.907

Table 3: Per-class F1 scores on the test set.

Check of the CNN design

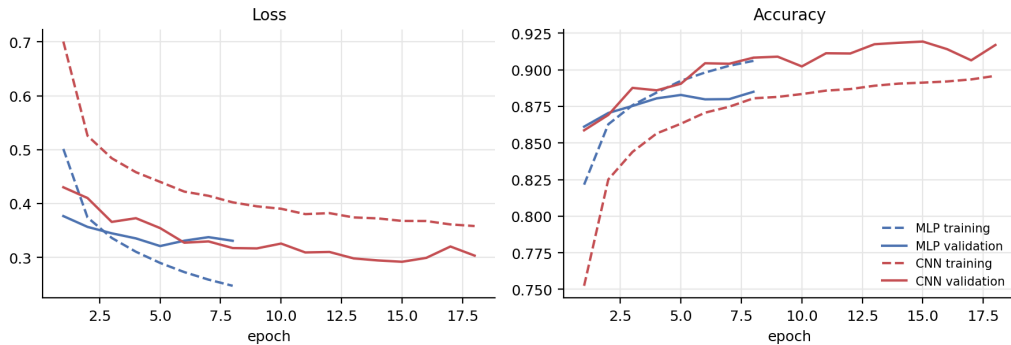


Figure 7: Training curves of the proposed CNN and a fully connected network (784–150–60–10), trained once with Adam, batch size 32 and early stopping (patience 3) on a 10% validation split of the training set. Test accuracy: CNN 90.7% (18 epochs), fully connected network 86.6% (8 epochs).

B How to run the code

The file `code.zip` contains `run_analysis.py` (data analysis, decision tree, bagging and random forest, noise check, SHAP values and Figures 1–5), `gradient_boosting.py` (the gradient boosting reference and Figure 6), `cnn_keras.py` (the check of the CNN design in Part 2), `requirements.txt` and a `README.md`. The data are loaded with `keras.datasets.fashion_mnist.load_data()`, which downloads Fashion-MNIST on the first run. To reproduce the results:

1. Install the packages with `pip install -r requirements.txt`.
2. From inside `code/`, run `python run_analysis.py` (about an hour on a laptop) and `python gradient_boosting.py` (about 90 minutes). They write all numbers to `results/` and all figures to `figures/`.
3. Optionally run `python cnn_keras.py` (needs TensorFlow, about one hour on a laptop CPU).

All random seeds are fixed (`random_state=42`), so the numbers in this report are reproduced with the package versions in `requirements.txt`.